

Go Concurrency Cheatsheet

Lessons: [13](#) — [Goroutines](#) · [14](#) — [Channels](#) · [15](#) — [Sync, Context & Patterns](#)

Examples: [13](#) · [14](#) · [15](#)

Covers: the complete API surface — builtins, `sync`, `sync/atomic`, `context`, `runtime`, `time`, `os/signal`, `testing`, `golang.org/x/sync` — plus the full pattern catalog

Legend: `[*]` = real Go API that the lessons have not covered yet

LAUNCHING / SCHEDULER

<code>go f(x)</code>	start f concurrently, caller continues
<code>go func(){...}()</code>	anonymous goroutine (closure captures vars)
<code>runtime.NumGoroutine()</code>	<code>[*]</code> how many goroutines alive (leak check)
<code>runtime.NumCPU()</code>	<code>[*]</code> logical CPU count
<code>runtime.GOMAXPROCS(n)</code>	<code>[*]</code> max OS threads running Go code (0 = read only)
<code>runtime.Gosched()</code>	<code>[*]</code> yield the processor to other goroutines
<code>runtime.Stack(buf, true)</code>	<code>[*]</code> dump stacks of ALL goroutines into buf
<code>debug.Stack()</code>	<code>[*]</code> runtime/debug: current goroutine stack as []byte
<code>debug.PrintStack()</code>	<code>[*]</code> print it to stderr
<code>runtime.LockOSThread()</code>	<code>[*]</code> pin goroutine to its OS thread (cgo/UI libs)
<code>runtime.UnlockOSThread()</code>	<code>[*]</code> release the pin
<code>GOMAXPROCS=4 go run .</code>	<code>[*]</code> same knob as env var

sync.WaitGroup

<code>wg.Add(n)</code>	+n to counter (call BEFORE `go`)
<code>wg.Done()</code>	-1 (always `defer wg.Done()`)
<code>wg.Wait()</code>	block until counter == 0
<code>wg.Go(f)</code>	<code>[*]</code> Go 1.25+: <code>Add(1) + go f() + Done()</code> in one call (zero value ready; pass as <code>*sync.WaitGroup</code> – never copy it)

CHANNELS: builtins

<code>make(chan T)</code>	unbuffered: send waits for a receiver
<code>make(chan T, n)</code>	buffered: blocks only when full / empty
<code>make(chan struct{})</code>	signal-only channel (zero-size value)
<code>ch <- v</code>	send
<code>v := <-ch</code>	receive
<code><-ch</code>	receive & discard (pure synchronization)
<code>v, ok := <-ch</code>	ok == false -> closed & drained
<code>close(ch)</code>	"no more values" (sender closes, exactly once)
<code>len(ch)</code>	values currently sitting in the buffer
<code>cap(ch)</code>	buffer capacity (0 for unbuffered)
<code>for v := range ch</code>	receive until closed & drained
<code>chan<- T</code>	send-only param type
<code><-chan T</code>	receive-only param type
<code>var ch chan T</code>	nil channel: send/receive block forever
<code>ch <- struct{}{}</code>	"ping" on a signal channel
<code>close(done)</code>	broadcast: every receiver unblocks at once

CHANNELS: select

```
select { case v := <-a: ...; case b <- x: ... }    first ready wins; random if several
default:                                         make the select non-blocking
case <-time.After(d):                          timeout branch
case <-ctx.Done():                             cancellation branch
case <-done:                                    stop-signal branch
ch = nil                                       disable a select case (nil blocks forever)
select {}                                       block the goroutine forever (deadlock if alone)
for { select { ... } }                         event loop shape
```

time: channel-shaped APIs

```
time.After(d)                                <-chan Time, fires once after d
time.Sleep(d)                                block this goroutine (never use to synchronize)
time.Tick(d)                                [*] repeating channel that can never be stopped – leaks
time.NewTimer(d)                             [*] *Timer with .C channel
    t.Stop()                                [*] cancel it -> false if it already fired
    t.Reset(d)                               [*] restart the timer
time.NewTicker(d)                             [*] *Ticker with .C repeating; `defer tk.Stop()`
    tk.Stop() / tk.Reset(d)                  [*] stop / re-interval the ticker
time.AfterFunc(d, f)                          [*] run f in a new goroutine after d; returns *Timer
time.Since(t0) / Until(t)                    [*] elapsed / remaining duration
```

sync.Mutex / sync.RWMutex

```
mu.Lock() / mu.Unlock()                      exclusive; pair with `defer mu.Unlock()`
mu.TryLock()                                [*] take it or return false, never blocks
rw.RLock() / rw.RUnlock()                   many concurrent readers
rw.Lock() / rw.Unlock()                     one writer, excludes all readers
rw.TryLock()                                [*] non-blocking write lock
rw.TryRLock()                               [*] non-blocking read lock
rw.RLocker()                                [*] RWMutex as a read-only sync.Locker
sync.Locker                                 [*] interface { Lock(); Unlock() } – accept either mutex
(zero value ready to use; never copy a mutex – `go vet` copylocks catches it)
```

sync.Once & lazy init

```
once.Do(f)                                  run f exactly once, ever, even under races
sync.OnceFunc(f)                            [*] -> func() that runs f once
sync.OnceValue(f)                           [*] -> func() T, computed once (lazy cached value)
sync.OnceValues(f)                          [*] -> func() (T, error), computed once
```

sync.Map (concurrent map) [*]

m.Store(k, v)	set
m.Load(k)	-> (value, ok)
m.LoadOrStore(k, v)	get existing or insert -> (actual, loaded)
m.LoadAndDelete(k)	get + remove atomically -> (value, loaded)
m.Delete(k)	remove
m.Swap(k, v)	replace -> (previous, loaded)
m.CompareAndSwap(k, old, new)	conditional replace -> bool
m.CompareAndDelete(k, old)	conditional delete -> bool
m.Range(func(k, v any) bool)	iterate; return false to stop
m.Clear()	remove everything

(prefer plain map + RWMutex unless keys are write-once / read-many)

sync.Pool [*]

pool := &sync.Pool{New: func() any {...}} factory for empty objects

pool.Get() reusable object (maybe fresh from New)

pool.Put(x) give it back – reset it first!

(cuts GC pressure for buffers; entries can vanish at any GC)

sync.Cond [*]

sync.NewCond(&mu)	condition variable over a Locker
c.Wait()	unlock, sleep until signalled, re-lock
c.Signal()	wake one waiter
c.Broadcast()	wake all waiters
c.L	the underlying Locker

(rare – a channel is usually simpler; always Wait inside a for-condition loop)

sync/atomic: typed atomics (preferred)

atomic.Int32 / Int64 / Uint32 / Uint64 / Uintptr / Bool / Pointer[T] / Value

a.Load()	read
a.Store(v)	write
a.Add(delta)	add -> new value (numeric types only)
a.Swap(v)	set -> old value
a.CompareAndSwap(old, new)	CAS -> swapped bool
a.And(mask) / a.Or(mask) [*]	atomic bitmask ops -> old value

atomic.Value [*] Load/Store/Swap/CompareAndSwap of one any-type
 (config hot-swap)

atomic.Pointer[T] [*] typed pointer swap without unsafe

sync/atomic: function forms (older style)

atomic.AddInt64(&n, 1)	add -> new
atomic.LoadInt64(&n)	read
atomic.StoreInt64(&n, v)	write
atomic.SwapInt64(&n, v)	set -> old
atomic.CompareAndSwapInt64(&n, old, new)	CAS -> bool

atomic.AndInt64 / OrInt64 [*] bitmask ops

(same set for Int32/Uint32/Uint64/Uintptr/Pointer; args must be *aligned* pointers)

context: create

context.Background()	root context (main, tests, top of a request)
context.TODO()	placeholder when you don't have one yet
context.WithCancel(p)	-> ctx, cancel ; always `defer cancel()`
context.WithTimeout(p, d)	-> ctx, cancel ; auto-cancel after d
context.WithDeadline(p, t)	-> ctx, cancel ; auto-cancel at time t
context.WithValue(p, k, v)	request-scoped value (use sparingly, typed key)
context.WithCancelCause(p)	[*] -> cancel(err) records a reason
context.WithTimeoutCause(p, d, e)	[*] timeout with a custom cause error
context.WithDeadlineCause(p, t, e)	[*] deadline with a custom cause error
context.WithoutCancel(p)	[*] keep values, drop cancellation (detached work)
context.AfterFunc(ctx, f)	[*] run f when ctx finishes; returns stop() bool

context: observe

<-ctx.Done()	channel closed on cancel / timeout
ctx.Err()	context.Canceled context.DeadlineExceeded
ctx.Value(key)	read a request-scoped value
ctx.Deadline()	[*] -> (time, ok)
context.Cause(ctx)	[*] the real reason (set by WithCancelCause)
context.CancelFunc	[*] type func()
context.CancelCauseFunc	[*] type func(cause error)

(ctx is always the FIRST param, named ctx; never store it in a struct)

os/signal: graceful shutdown [*]

signal.NotifyContext(ctx, os.Interrupt, syscall.SIGTERM)	-> ctx, stop ; ctx dies on signal
signal.Notify(ch, sigs...)	deliver signals into your own channel
signal.Stop(ch)	stop delivering to that channel
signal.Ignore(sigs...) / signal.Reset(sigs...)	ignore / restore default
srv.Shutdown(ctx)	http.Server: drain in-flight requests, then stop

golang.org/x/sync/errgroup [*]

errgroup.WithContext(ctx)	-> g, ctx (ctx cancels on the first error)
var g errgroup.Group	zero value works (no cancellation)
g.Go(func() error {...})	run a task in the group
g.TryGo(f)	start only if under the limit -> bool
g.SetLimit(n)	cap concurrent goroutines (-1 = unlimited)
g.Wait()	wait for all, return the first non-nil error

(the modern replacement for WaitGroup + error channel)

golang.org/x/sync: semaphore & singleflight [*]

semaphore.NewWeighted(n)	counting semaphore of weight n
sem.Acquire(ctx, w)	block until w free (or ctx dies) -> error
sem.TryAcquire(w)	non-blocking -> bool
sem.Release(w)	give the weight back
singleflight.Group	collapse duplicate concurrent work
g.Do(key, fn)	-> (v, err, shared) – one call per key at a time
g.DoChan(key, fn)	same, but returns <-chan Result
g.Forget(key)	stop sharing the in-flight result for key

TESTING & DEBUGGING CONCURRENCY

```
go run -race . / go test -race ./...    data-race detector
go test -cpu=1,4 / -parallel=n  [*] vary GOMAXPROCS / parallel test count
t.Parallel()                    [*] mark test as parallel-safe
t.Context()                     [*] Go 1.24+: ctx cancelled when the test ends
b.RunParallel(func(pb *testing.PB){ for pb.Next() {...} })  [*] parallel benchmark
b.SetParallelism(p)            [*] goroutines per CPU in RunParallel
syncstest.Test(t, f)           [*] testing/syncstest: run f in a fake-clock bubble
syncstest.Wait()               [*] block until every goroutine in the bubble is idle
pprof.Lookup("goroutine").WriteTo(w, 1)  [*] dump all goroutine stacks
import _ "net/http/pprof"      [*] serves /debug/pprof/goroutine live
trace.Start(w)/trace.Stop()    [*] runtime/trace: scheduler timeline
kill -QUIT <pid>               [*] SIGQUIT prints all goroutine stacks and exits
```

PATTERNS: running work

Worker pool	N goroutines range over one jobs channel; results channel out
Fan-out	several goroutines read from the SAME channel to parallelize
Fan-in	merge many channels into one; WaitGroup closes the output
Scatter-gather	fan-out then fan-in – the pair, as one operation
Semaphore chan	make(chan struct{}, n); acquire = send, release = receive the ctx-aware form: select on the send vs <-ctx.Done()
Generator	func gen() <-chan T { ch := make(...); go ...; return ch }
Bounded take	read at most n from an infinite generator, then cancel it
Pipeline	stage(in <-chan T) <-chan U; chained; ctx cancels every stage each stage owns its output channel and closes it
Future/promise	ch := make(chan Result, 1); go func(){ ch <- work() }() BUFFERED, so the goroutine never blocks if nobody reads
First wins	race N sources; buffered chan of size N, take the first result (cancel the rest; unbuffered here leaks every loser)
Tee channel	[*] split one input stream into two independent output streams
Bridge channel	[*] flatten a <-chan (<-chan T) into one stream
Batching	[*] accumulate n items or wait d, then flush – one write per batch

PATTERNS: coordination & lifecycle

done channel	close(done) broadcasts "stop" to every receiver at once
quit in a select	for { select { case <-quit: return; case j := <-jobs: ... } }
cancellable loop	the same shape with <-ctx.Done() – the standard worker body
or-channel	combine several done signals into one: fire when ANY fires
or-done wrapper	forward values until ctx/done fires, then return – lets a downstream stage exit without draining its input
Barrier	everyone waits for everyone: wg.Wait(), or close a start chan to release all goroutines at the same instant
Heartbeat	[*] a goroutine emits on a ticker so the supervisor knows it lives – the fix for "is it stuck or just slow?"
errgroup	first error cancels the ctx; g.Wait() returns that error g.SetLimit(n) makes it a bounded worker pool too
Context tree	cancel a parent -> every child ctx cancels, depth-first
Deadline budget	one parent timeout, shorter per-task timeouts beneath it
Interruptible sleep	select { case <-time.After(d): case <-ctx.Done(): } – time.Sleep cannot be cancelled; this can
Graceful shutdown	NotifyContext -> cancel -> srv.Shutdown -> wg.Wait() -> exit
Guaranteed exit	every goroutine needs a path that ends it – the leak rule

PATTERNS: sharing state

Confinement	one goroutine owns the data; others talk to it via channels
Single writer	one writer, many readers -> RWMutex or an atomic snapshot
Mutex or channel	mutex for STATE, channel for HANDOFF. If you find yourself guarding a channel with a mutex, you picked wrong.
Read-through cache	RWMutex: RLock to look up, Lock to fill on a miss
Double-checked	RLock-check, then Lock-recheck-create – the get-or-create race (or LoadOrStore, which does it atomically)
Lazy singleton	once.Do, or sync.OnceValue for a cached computed value
CAS loop	for { old := a.Load(); new := f(old) if a.CompareAndSwap(old, new) { break } } lock-free max/min/accumulate
Atomic snapshot	atomic.Pointer[T] + copy-on-write: readers never block (the full treatment is on the low-latency sheet)
Sharded map	[*] N maps + N mutexes keyed by hash(k)%N – the hot-map fix
Pub/sub	one publisher, many subscriber channels; drop or disconnect the slow ones – never block the publisher
Hub	one goroutine owns the client set; register/unregister/broadcast channels feed it (the full shape is on the real-time sheet)

PATTERNS: rate & duplicate work

Rate limiter	<-time.Tick(d), or a NewTicker per worker (and Stop it)
Token bucket	a buffered chan refilled by a ticker; take = receive (x/time/rate does this properly – see the resilience sheet)
Singleflight	collapse concurrent identical calls into one; the rest share the result – the cache-stampede fix
Debounce	[*] reset a timer on every event; act only after d of silence
Throttle	[*] act at most once per interval, dropping what arrives between

PATTERNS THAT LIVE ON OTHER SHEETS

copy-on-write, false sharing, sharding	-> 46-48 low-latency
circuit breaker, bulkhead, hedged requests	-> 36-68 resilience
retry with backoff + jitter, load shedding	-> 36-68 resilience
the hub, backpressure, per-user state	-> 58-67 real-time
outbox, saga, idempotent consumer	-> 34-35-44 events

PANICS, DEADLOCKS & LEAKS (memorize)

send on a closed channel	panic
close of a closed channel	panic
close of a nil channel	panic
receive from a closed channel	OK – zero value, ok == false
send/receive on a nil channel	blocks forever
unbuffered send, no receiver	"all goroutines are asleep – deadlock!"
wg.Add() inside the goroutine	races with Wait()
copying a Mutex/WaitGroup	silently broken – go vet copylocks
missing defer cancel()	leaks the context's timer/goroutine
receiver closing the channel	panics the sender – only the sender closes
goroutine blocked forever	leak: every goroutine needs a guaranteed exit
unbounded `go` per request	memory blowup – use a pool or SetLimit